

Universidad Nacional de Lomas de Zamora

Facultad de Ingeniería

Proyecto FINAL - Estacion de calidad por Perfilometria

Alumno: QUINTANA, Fernando Miguel

In []:

```
1  # Importamos Librerías
2  import cv2
3  import numpy as np
4  import glob
5
6  # Definimos Las dimensiones de La tabla de ajedrez de prueba
7  CHECKERBOARD = (6, 8) # 6 puntos verticales y 8 puntos horizontales
8
9  # Se establece criterio máximo de iteración y error
10 criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.0001)
11
12 # Vector para almacenar cada vector 3D en La proyección de La imagen
13 objpoints = []
14
15 # Vector para almacenar cada vector 2D en La proyección de La imagen
16 imgpoints = []
17
18 # Se definen Las coordenadas del espacio para Los puntos en 3D
19 objp = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.float32)
20 objp[0, :, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.reshape(-1, 2)
21 prev_img_shape = None
22
23 # Carpeta donde se encuentran Las fotos de calibración
24 images = glob.glob("Imagenes/*.jpg")
25 cont = 0
26
27 # Se itera cada imagen
28 for fname in images:
29     # Se Lee imagen
30     img = cv2.imread(fname)
31
32     # Se convierte a escalas de grises
33     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
34
35     # Se identifican Las esquinas de La tabla de ajedrez
36     # Si Las encuentra, devuelve True
37     ret, corners = cv2.findChessboardCorners(
38         gray,
39         CHECKERBOARD,
40         cv2.CALIB_CB_ADAPTIVE_THRESH
41         + cv2.CALIB_CB_FAST_CHECK
```

```
42         + cv2.CALIB_CB_NORMALIZE_IMAGE,
43     )
44
45     if ret:
46         # Agrega los puntos al vector
47         objpoints.append(objp)
48
49         # Refina los puntos para los vectores 2D
50         corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
51
52         # Se agregan a los puntos de imagen
53         imgpoints.append(corners2)
54
55         # Dibuja las esquinas sobre la imagen
56         img = cv2.drawChessboardCorners(img, CHECKERBOARD, corners2, ret)
57
58         # Guarda las imágenes en la carpeta nuevamente
59         # cv2.imwrite(str("Fotos/") + str(cont) + ".jpg", img)
60         cont += 1
61
62         # Muestra las imágenes
63         cv2.imshow("img", img)
64         cv2.waitKey(10)
65
66     # Cierra todas las ventanas
67     cv2.destroyAllWindows()
68
69     # Obtiene el tamaño de imagen
70     h, w = img.shape[:2]
71
72     # Calibra la cámara pasando como parámetros, los diferentes vectores 3D y 2D de las imágenes procesadas
73     ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
74         objpoints, imgpoints, gray.shape[:-1], None, None
75     )
76
77     # Calcula el error
78     mean_error = 0
79     for i in range(len(objpoints)):
80         imgpoints2, _ = cv2.projectPoints(objpoints[i], rvecs[i], tvecs[i], mtx, dist)
81         error = cv2.norm(imgpoints[i], imgpoints2, cv2.NORM_L2) / len(imgpoints2)
82         mean_error += error
83
```

```
84 # Muestra los datos
85 print("Total error: " + str(mean_error / len(objpoints)))
86 print("Camera matrix:\n", mtx)
87 print("dist:\n", dist)
88 print("rvecs:\n", rvecs)
89 print("tvecs:\n", tvecs)
90
91 # Calibra todas las imágenes de la carpeta para obtener el resultado final
92 newcameramt, roi = cv2.getOptimalNewCameraMatrix(
93     mtx, dist, (w, h), 1, (w, h), centerPrincipalPoint=False
94 )
95 cont = 0
96 for fname in images:
97     img = cv2.imread(fname)
98
99     # Corrige la distorsión
100     dst = cv2.undistort(img, mtx, dist, None, newcameramt)
101
102     # Recorta la imagen
103     x, y, w, h = roi
104     dst = dst[y:y+h, x:x+w]
105
106     cont += 1
107     cv2.imwrite(str(cont) + ".jpg", dst)
108
```

In []:

1